FINAL PROJECT REPORT

PROJECT FORESIGHT

Demand Forecasting and Inventory Intelligence Platform

NorthBay Living - Data Science & Analytics Engagement

Prepared by

D.Kundhana | B.Sanjay Kumar | Pratikhya Sahoo

Team No. 5

July 2026

GitHub Repository | Live Dashboard | Scoring API

00 Executive Summary

A decision-focused summary of the delivered forecasting and inventory-intelligence system.

Project FORESIGHT converts operational sales, product, and inventory data into weekly SKU-level forecasts, stockout and overstock risk scores, financial impact estimates, and recommended inventory actions. The solution was developed for NorthBay Living, a direct-to-consumer home and lifestyle business that needs to reduce lost sales from stockouts while avoiding working capital tied up in slow-moving inventory.

INR 793,250.16 **Potential lost revenue** stockout exposure	**INR 1,128,180.40** **Recommended replenishment cost** MOQ-rounded orders	**INR 6,347.14** **Excess inventory value** overstock exposure
8 weeks **Forecast horizon** 30 Dec 2024 to 17 Feb 2025	**150 SKUs** **Scored products** 1,200 future SKU-week rows	**39.51%** **Selected model WAPE** 8.37 points better than prior-week naive

The final system includes a reproducible Python pipeline, leakage-safe feature engineering, multiple forecasting baselines, HistGradientBoosting modelling, rolling-origin cross-validation, an 8-week recursive future forecast, transparent risk logic, a Streamlit planning dashboard, and a publicly deployed FastAPI scoring service.

Executive action: Expedite and replenish high stockout-risk SKUs first; reduce, transfer, promote, or markdown high overstock-risk SKUs; refresh the forecast and risk outputs every week before purchasing decisions.

00 Contents

The report is organised around the project-report submission requirement: overview, technology stack, architecture and screenshots, results, and conclusion.

01 Project Overview and Business Context

02 Objectives, Scope, and Success Criteria

03 Technology Stack and Repository Structure

04 Solution Architecture

05 Data Sources and Data Quality

06 Exploratory Analysis and Business Insights

07 Feature Engineering and Forecasting Method

08 Model Validation and Performance

09 Eight-Week Future Forecast

10 Risk Scoring and Decisioning

11 Planning Dashboard

12 FastAPI Scoring Service and Deployment

13 Testing, Reproducibility, and Governance

14 Business Impact and Recommendations

15 Limitations and Future Enhancements

16 Conclusion

17 References and Appendices

01 Project Overview and Business Context

Why the project was built and how it supports inventory decisions.

NorthBay Living sells home and lifestyle products online and manages inventory from operational exports and spreadsheets. The business problem is two-sided: popular products can run out, causing lost sales and poor customer experience, while slow-moving products can remain in stock, locking working capital and increasing markdown risk.

The engagement required a practical analytics product rather than a notebook-only analysis. Project FORESIGHT therefore links forecasting to operational decisioning and exposes the output through tools that the operations and finance teams can use directly.

1.1 Problem statement

- **Stockout problem:** insufficient inventory causes lost sales and urgent replenishment.
- **Overstock problem:** excess inventory locks cash and can lead to carrying cost, markdowns, and obsolescence.
- **Planning problem:** manual judgement and spreadsheets do not consistently combine demand history, promotions, lead time, on-hand stock, and on-order stock.
- **Usability problem:** decision-makers need a prioritised interface, not only model outputs in code or notebooks.

1.2 Delivered capability

1. Generate weekly SKU-level demand forecasts over an 8-week horizon.
2. Compare forecasting models against naive baselines using time-aware validation.
3. Score stockout and overstock risk for every SKU.
4. Attach recommended actions and INR value-at-stake to each SKU.
5. Serve the results through a filterable dashboard and a public scoring API.

02 Objectives, Scope, and Success Criteria

Alignment between the engagement brief and the implemented solution.

Objective	How Project FORESIGHT addresses it
Beat a naive baseline	HistGradientBoosting achieved 39.51% WAPE versus 47.88% for the previous-week naive and 53.30% for the 52-week seasonal naive.
Flag stockout and overstock risk	Every scored SKU receives risk levels, risk scores, recommended order quantity, value at stake, and an action.
Quantify business impact	Outputs include potential lost revenue, excess inventory value, and recommended replenishment cost in INR.
Provide a usable interface	The Streamlit dashboard includes filters, executive KPIs, risk priorities, model performance, SKU Explorer, and downloads.
Deploy a scoring service	The FastAPI application exposes forecast, score, summary, search, and top-risk endpoints through a public Render URL.
Ensure reproducibility	The repository separates pipeline, features, models, reports, application, API, and tests, with module-based run commands.

2.1 In scope

- Weekly forecasting at SKU level over eight future weeks.
- Stockout and overstock scoring based on forecast demand and inventory position.
- Reorder recommendations rounded to minimum order quantity.
- Financial impact estimation and prioritisation.
- Streamlit dashboard, FastAPI service, documentation, and executive readout.

2.2 Out of scope

- Live integration with enterprise systems.
- Real-time streaming pipelines.
- Automated purchase-order placement.
- Price optimisation, supplier selection, or a full mathematical inventory optimiser.

03 Technology Stack and Repository Structure

Tools used to build, evaluate, deploy, and document the system.

Layer	Technology	Purpose
Programming and data	Python 3.11; pandas; NumPy	Core processing, cleaning, aggregation, features, and metrics.
Machine learning	scikit-learn; joblib	Forecasting models, preprocessing pipelines, evaluation, and model persistence.
Visualisation	Plotly; Matplotlib	Interactive dashboard visualisations and analysis charts.
Dashboard	Streamlit	Non-technical planning interface and downloadable reports.
API	FastAPI; Uvicorn; Pydantic	Public scoring service, validation, and interactive Swagger documentation.
Quality and versioning	pytest; Git; GitHub	Automated checks, source management, and reproducible delivery.
Deployment	Streamlit Community Cloud; Render	Public hosting for the dashboard and scoring API.

3.1 Repository structure

```
project-foresight/
|- app/dashboard.py
|- data/{raw, processed}/
|- models/ and reports/
|- service/main.py
|- src/{pipeline, features, baseline, model, rolling_cv,
|      future_forecast, risk_scoring}.py
|- tests/
`- requirements.txt and README.md
```

This structure keeps raw data, processed outputs, model artefacts, reports, stakeholder applications, and tests separate. It also supports end-to-end reruns without manual spreadsheet editing.

04 Solution Architecture

How operational extracts become forecasts, risk scores, and stakeholder-facing products.

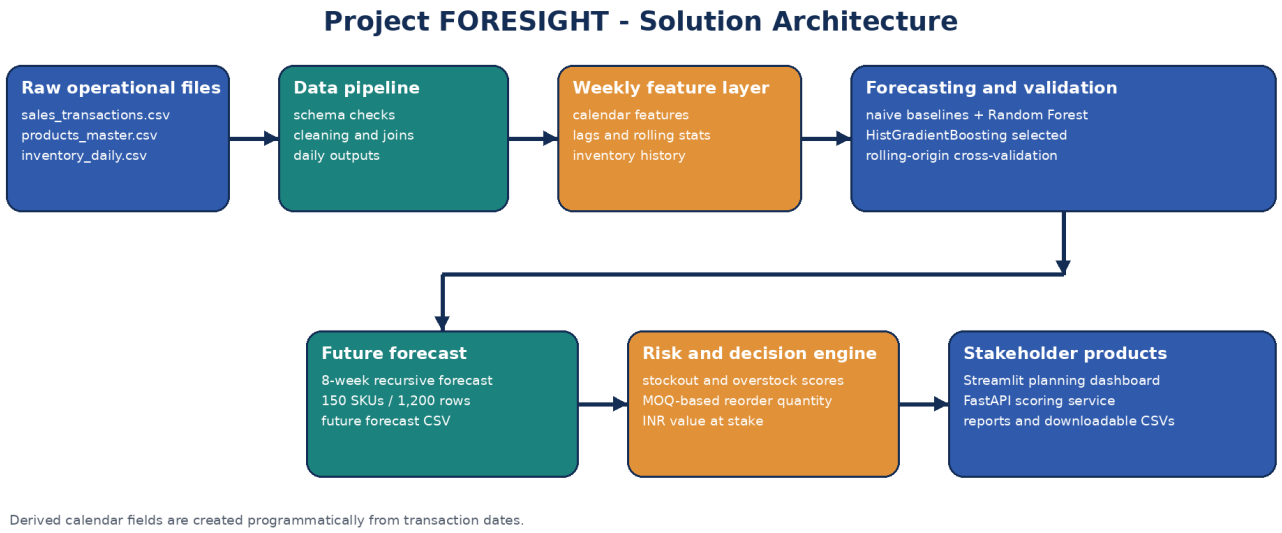

Figure 1. Project FORESIGHT solution architecture.

The implementation ingests three raw operational files and derives calendar and seasonal features programmatically from transaction dates. The feature layer feeds baseline models and candidate machine-learning models. The selected model is then used recursively to create an 8-week future forecast. Finally, the risk engine combines forecast demand with inventory and product economics to generate decisions that are served through the dashboard and API.

4.1 Main processed outputs

Output	Purpose
sales_daily.csv	Clean daily sales and revenue by SKU.
sku_master.csv	One validated product record per SKU.
inventory_snapshots.csv	Daily on-hand, on-order, stockout, and lost-sales signals.
weekly_demand.csv	Weekly modelling dataset with leakage-safe features.
model_predictions.csv	Final-test actuals and benchmark/model predictions.
future_8_week_forecast.csv	True 8-week recursive forecasts for every SKU.
inventory_risk_scores.csv	Risk scores, actions, order quantities, and financial impact.

05 Data Sources and Data Quality

Input data, cleaning decisions, validation rules, and analysis-ready scale.

Raw file	Key fields	Cleaning and validation
sales_transactions.csv	Invoice, date, SKU, quantity, price, channel, revenue	Trim identifiers; parse dates and numerics; remove cancellations, missing identifiers, invalid dates, non-positive quantities, and non-positive prices; recompute revenue.
products_master.csv	SKU, description, category, cost, price, lead time, shelf life, supplier, MOQ	Enforce unique SKU; trim labels; validate positive numeric fields; derive launch date from valid sales.
inventory_daily.csv	Date, SKU, stock, on-order, lost sales, stockout, reorder point	Parse dates and numerics; reject missing or negative values; enforce one date-SKU record.

5.1 Data-quality controls

- Required-column checks fail early when an input schema is incomplete.
- Many-to-one joins are validated to prevent accidental row multiplication.
- Date-SKU uniqueness is checked in daily sales and inventory outputs.
- Missing, invalid, and negative values are rejected where they would invalidate modelling or risk calculations.
- Predicted demand is clipped at zero because negative unit demand is not operationally meaningful.

109,650
Daily SKU records
analysis-ready scale

15,900
Weekly SKU records
model dataset

104
Complete weeks
time-series history

150
Unique SKUs
scored products

1,200
Future rows
8 weeks x 150 SKUs

3 files
Raw sources
calendar derived from dates

06 Exploratory Analysis and Business Insights

Patterns that informed feature engineering, model selection, and inventory policy.

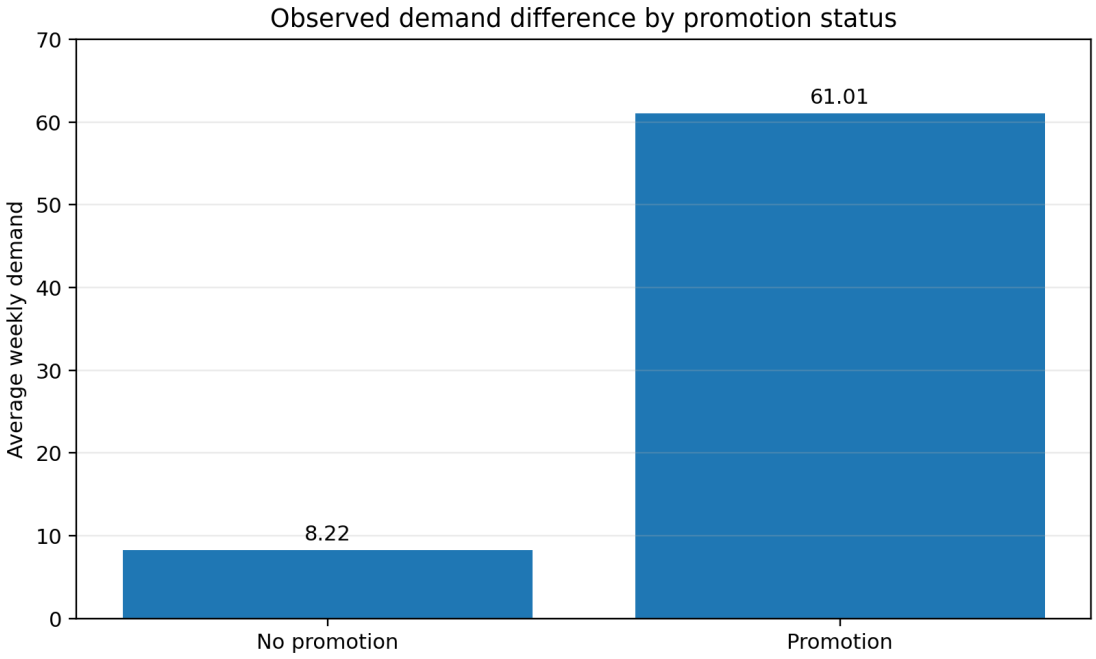

Figure 2. Observed average weekly demand with and without promotion.

- **Promotion association:** average demand was 61.01 during promotion periods versus 8.22 without promotion, an observed difference of about 642%.
- **Demand heterogeneity:** demand varies widely across SKUs and includes intermittent or zero-demand weeks, so a single reorder rule is unsuitable.
- **Inventory pressure:** on-hand, on-order, stockout days, and lost sales contain operational signals that complement demand history.
- **Dead stock:** the strict dead-stock rule identified no products meeting every criterion in the current dataset.
- **Business implication:** planning should prioritise high-demand, high-value, and high-stockout products while treating slow movers with more conservative replenishment.

Causality warning: The promotion result is an observational association. It does not prove that promotion alone caused the full demand increase.

07 Feature Engineering and Forecasting Method

Leakage-safe signals, candidate models, and fair time-series evaluation.

7.1 Leakage-safe features

Feature group	Examples
SKU identity	One-hot encoded SKU identifier.
Calendar and seasonality	ISO year/week, month, quarter, sine/cosine seasonal encodings.
Promotion schedule	Current promotion flag and previous-week promotion flag.
Demand lags	1, 2, 4, 8, 13, 26, and 52-week lags.
Rolling behaviour	4, 8, and 13-week means and standard deviations; recent zero-demand rate.
Inventory history	Previous-week on-hand, on-order, stockout days, and lost sales.

Every lag and rolling feature is built using information available before the forecasted week. This prevents future demand from leaking into model inputs.

7.2 Candidate models and baselines

- **Previous-week naive:** uses the most recent observed week as the forecast.
- **Seasonal naive:** uses demand from 52 weeks earlier.
- **Random Forest:** tree ensemble benchmark for nonlinear interactions.
- **HistGradientBoosting:** selected model using Poisson loss for non-negative count-like demand.

7.3 Validation design

Model selection used chronological splits rather than random sampling. The final model was evaluated on a held-out test period and additionally checked using rolling-origin cross-validation with four expanding folds and an 8-week horizon per fold. WAPE was the primary metric, with MAE, RMSE, and bias as secondary diagnostics.

08 Model Validation and Performance

Evidence that the selected model improves on the operational baselines.

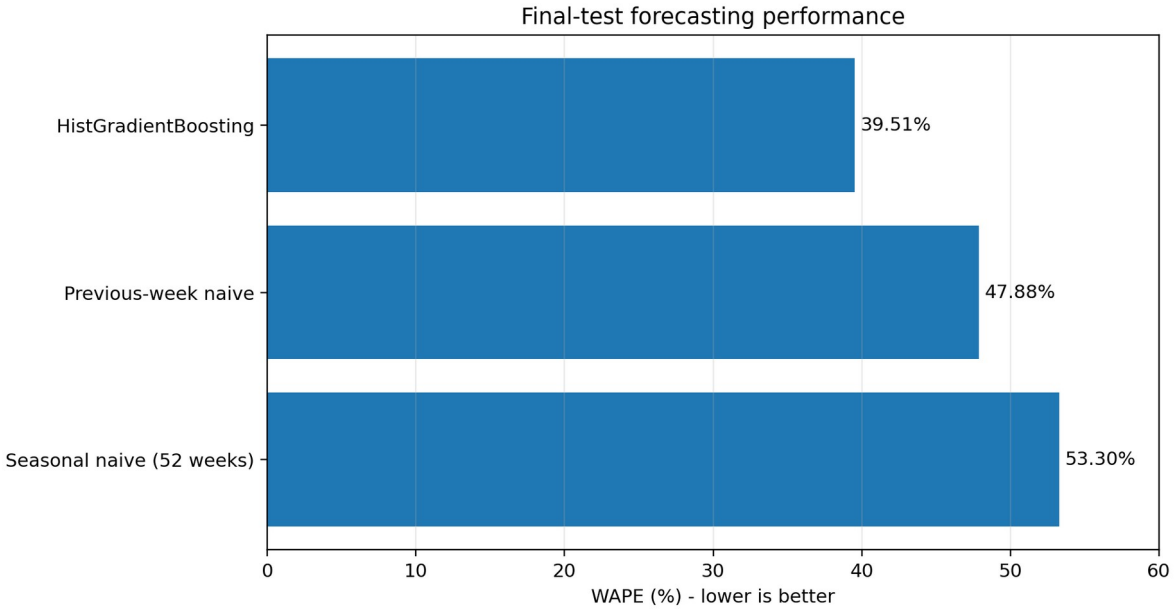

Figure 3. Final-test WAPE comparison; lower values indicate better aggregate accuracy.

Model	WAPE	MAE	RMSE	Bias	Rank
HistGradientBoosting	39.51%	57.01	96.10	2.94%	1
Previous-week naive	47.88%	69.08	121.54	-1.05%	2
Seasonal naive - 52 weeks	53.30%	76.89	143.16	0.23%	3

- HistGradientBoosting improved WAPE by 8.37 percentage points compared with the previous-week naive baseline.
- Rolling-origin cross-validation produced 44.58% WAPE for the selected model compared with 51.16% for the previous-week naive baseline.
- Positive test bias of 2.94% indicates slight aggregate over-forecasting during the final test period.
- SKU-level accuracy remains uneven; risk scoring therefore also uses safety stock, inventory position, lead time, and value-at-stake logic.

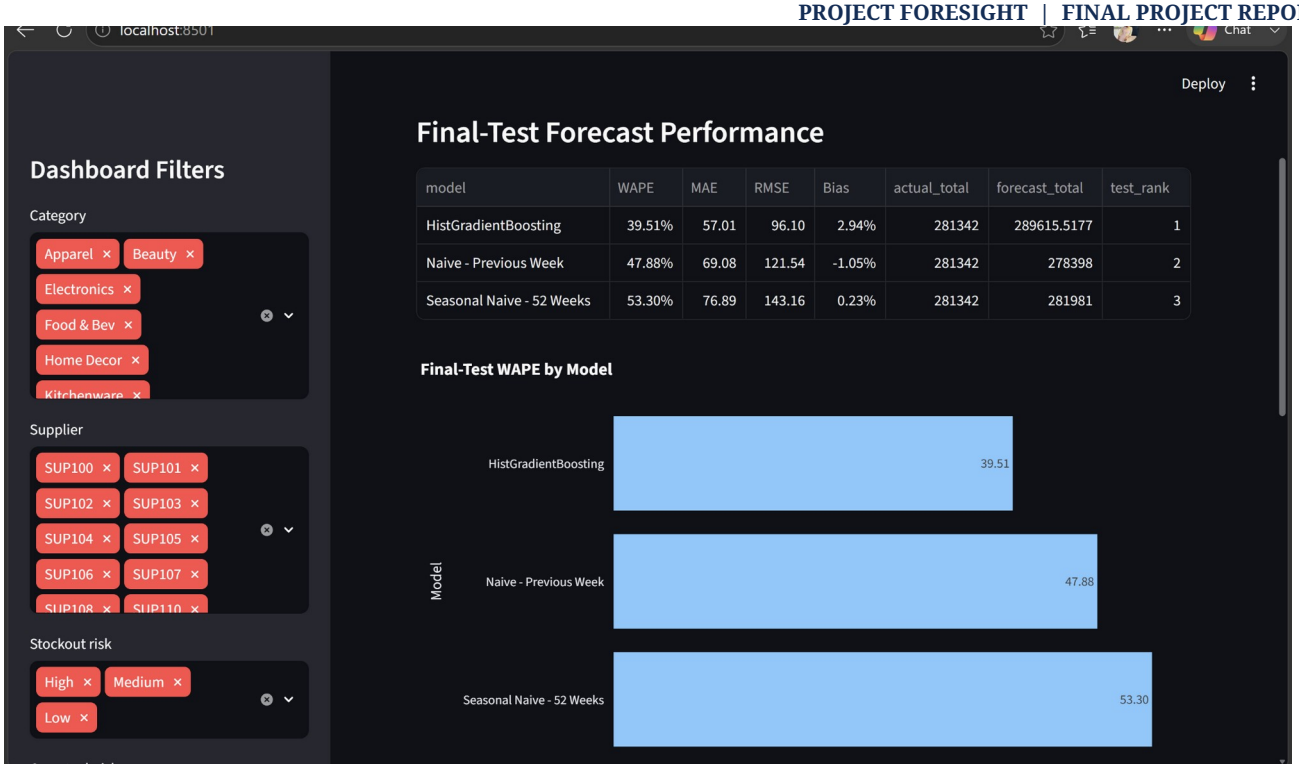

Figure 4. Forecast-performance page in the Streamlit dashboard.

09 Eight-Week Future Forecast

Recursive forecasting beyond the historical test period.

30 Dec 2024	**17 Feb 2025**	**8 weeks**
Forecast start	**Forecast end**	**Horizon**
first future week	eighth future week	recursive generation
150 SKUs	**1,200**	**136,358.05 units**
Products	**Output rows**	**Forecast demand**
complete coverage	SKU-week forecasts	sum across horizon

The future-forecast module loads the fitted model, creates each next week in sequence, predicts non-negative demand, appends the prediction to the history, and then rebuilds lag and rolling features for the following week. This recursive procedure ensures that later future weeks use earlier forecasts rather than unavailable actual demand.

Operational use: The future forecast is the demand input for current stockout, overstock, financial-impact, and replenishment calculations.

10 Risk Scoring and Decisioning

Transparent rules that convert forecasts into inventory actions.

10.1 Core calculations

Calculation	Simplified definition
Inventory position	Ending on-hand units + ending on-order units.
Lead-time demand	Weekly forecast x lead-time weeks.
Safety stock	1.65 x 13-week demand standard deviation x square root of lead-time weeks.
Stockout gap	Amount by which inventory position falls below lead-time demand plus safety stock.
Overstock excess	Inventory above the 8-week forecast requirement plus safety stock.
Recommended order	Target inventory minus inventory position, rounded upward to the SKU minimum order quantity.
Potential lost revenue	Stockout gap x list price.
Excess inventory value	Excess units x unit cost.

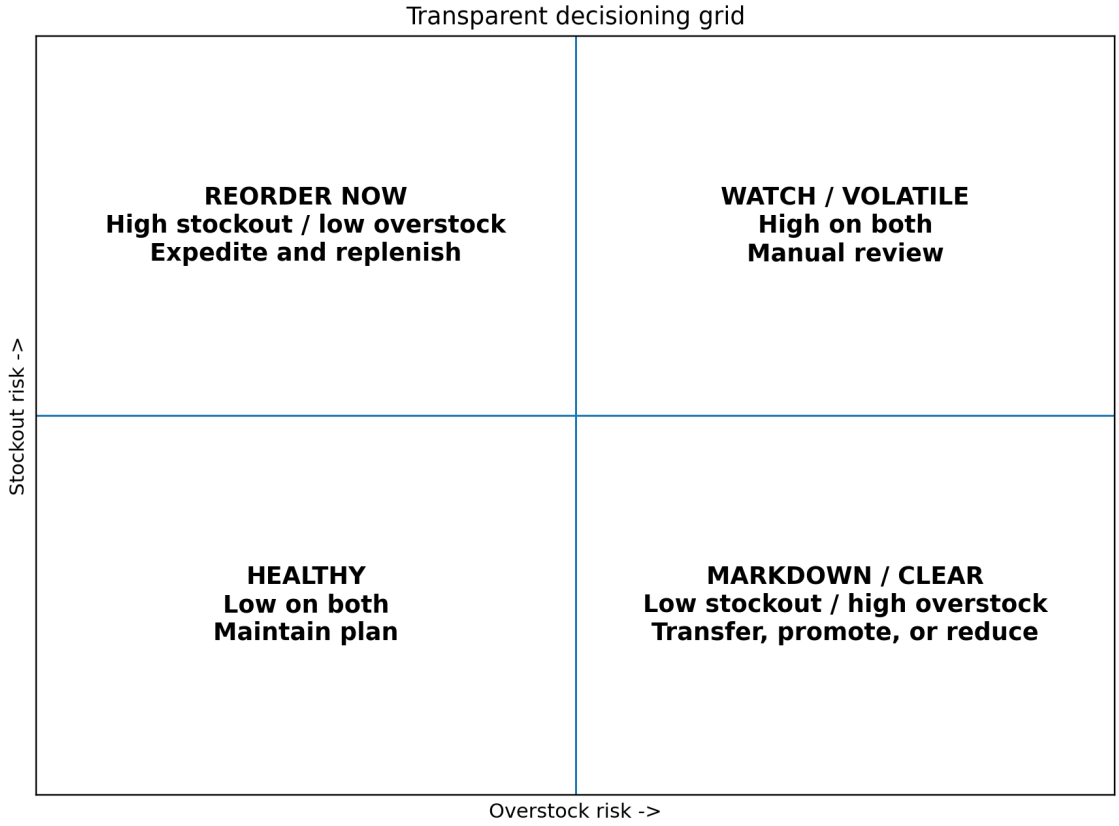

Figure 5. Decisioning grid that maps risk combinations to actions.

10.2 Risk results

Risk type	High	Medium	Low
Stockout risk	9	93	48
Overstock risk	3	8	139

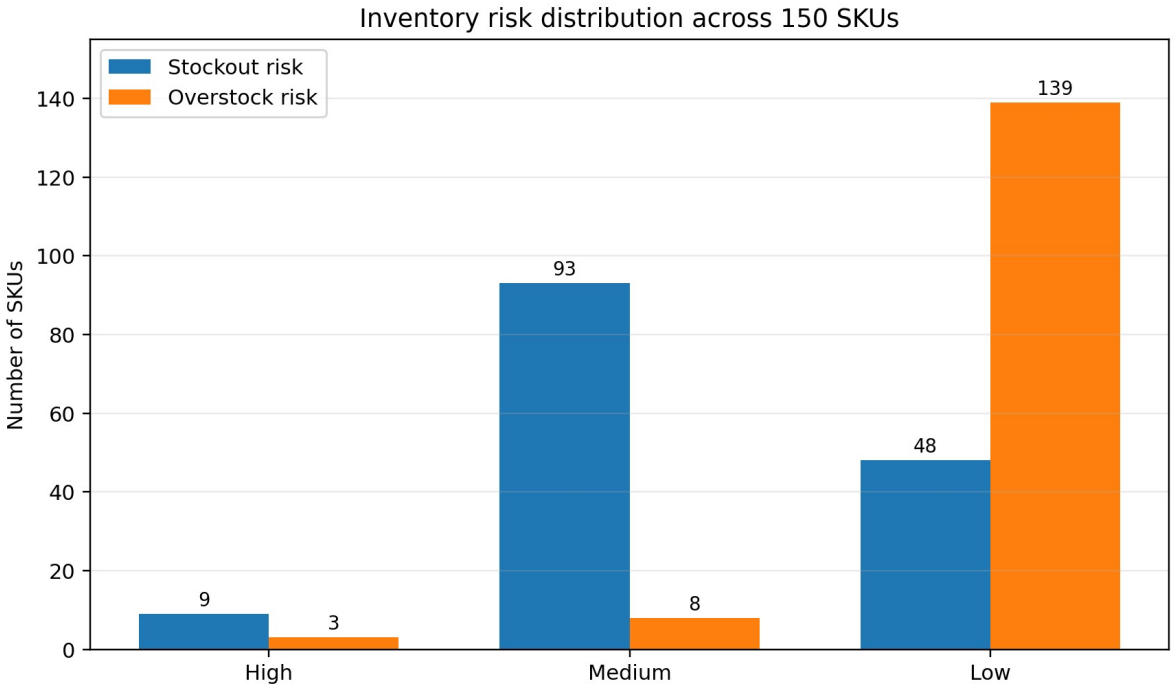

Figure 6. Risk distribution across the 150 scored SKUs.

- High stockout risk: expedite supply and place the recommended replenishment order.
- Medium stockout risk: review supplier timing and place or expedite the recommended order.
- High overstock risk: pause replenishment and consider transfer, promotion, or markdown.
- Low risk: maintain the current plan and monitor weekly.

11 Planning Dashboard

A usable planning interface for operations and finance stakeholders.

The Streamlit dashboard is designed for non-technical users and includes category, supplier, stockout-risk, overstock-risk, and SKU filters. It separates executive KPIs from detailed risk and forecast views while allowing filtered CSV downloads.

11.1 Main dashboard pages

- Executive Overview - headline KPIs, risk distributions, and replenishment cost.
- Risk Priorities - ranked stockout and overstock exposure with action lists.
- Forecast Performance - final-test metrics and rolling-origin validation.
- SKU Explorer - one-SKU forecast, risk details, inventory context, and action.
- Methodology - source files, assumptions, formulas, and limitations.

Live dashboard: https://project-foresight-szbpqvrcxp8x6sbrf8qmzx.streamlit.app/

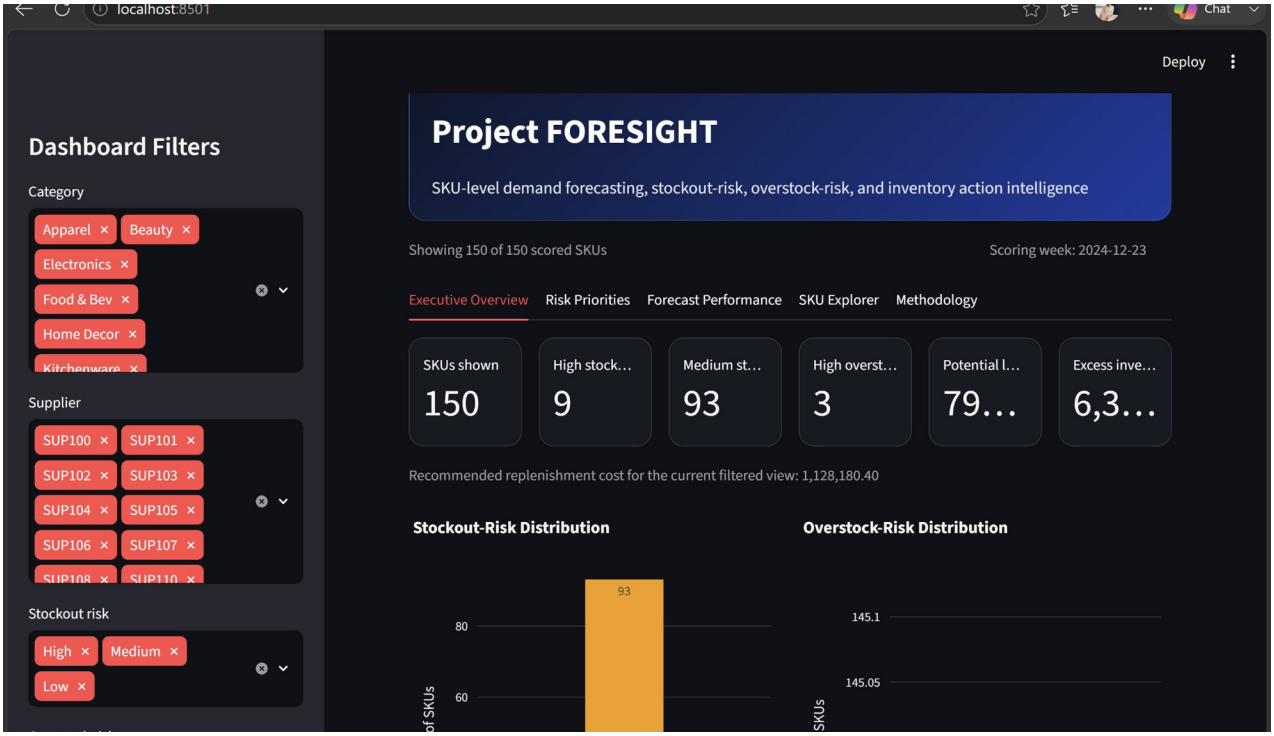

Figure 7. Executive Overview with filters, risk counts, and financial exposure.

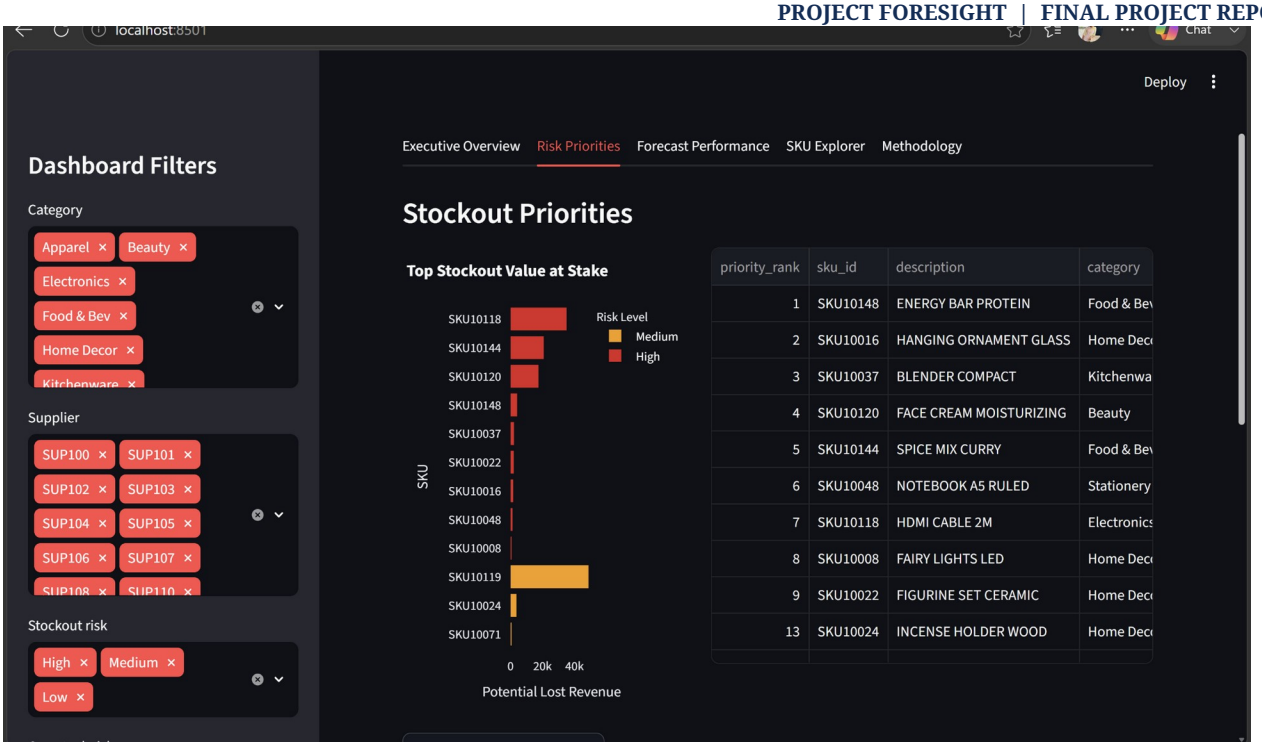

Figure 8. Stockout Priorities page with ranked value-at-stake and SKU table.

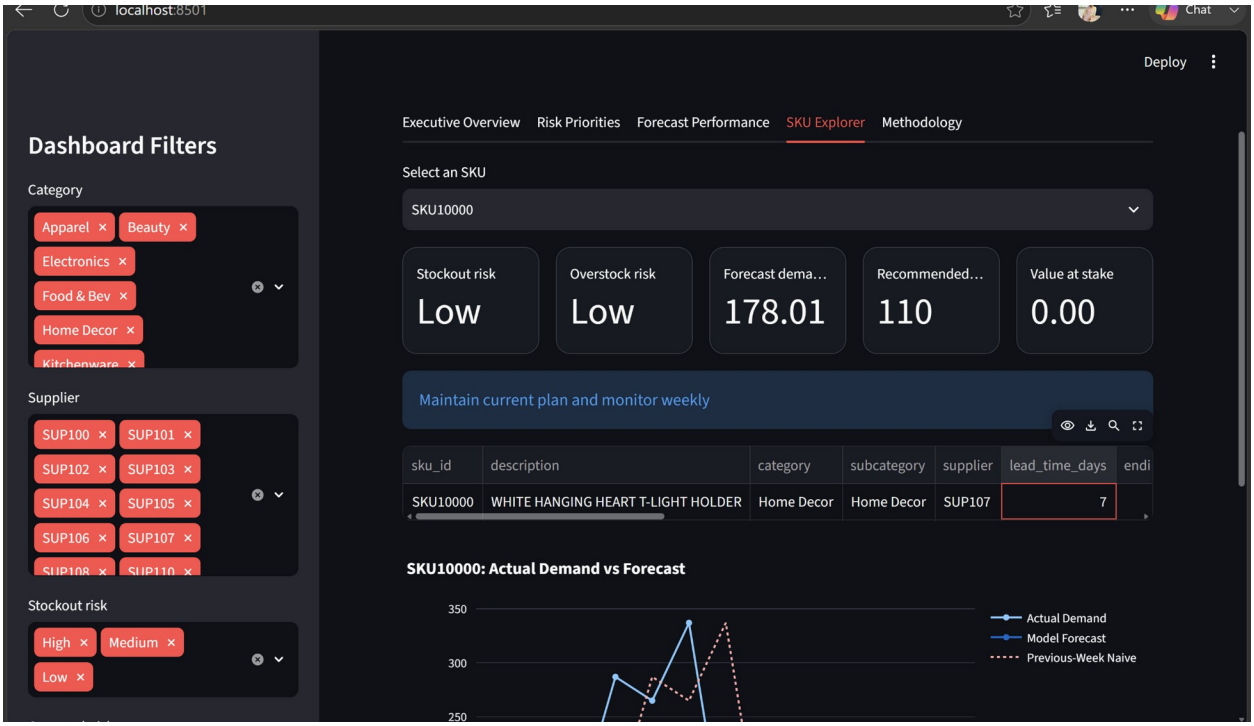

Figure 9. SKU Explorer showing risk, forecast demand, reorder quantity, action, and actual-versus-forecast history.

12 FastAPI Scoring Service and Deployment

A public service for forecast retrieval, risk intelligence, and what-if scoring.

Method	Endpoint	Purpose
GET	/	Service metadata and links.
GET	/health	Checks output-file availability and service status.
GET	/summary	Returns the executive risk summary.
GET	/skus	Searches and paginates scored SKUs.
GET	/forecast/{sku_id}	Returns the 8-week future forecast for one SKU.
GET	/score/{sku_id}	Returns saved risk intelligence and future forecast.
POST	/score	Recalculates one SKU using optional inventory and commercial overrides.
GET	/top-risks	Returns the highest-priority stockout, overstock, or overall risks.

The API validates requests with Pydantic, returns 404 for unknown SKUs, returns 422 for invalid request values, and returns a clear service error when required output files are missing. Swagger documentation is available at /docs.

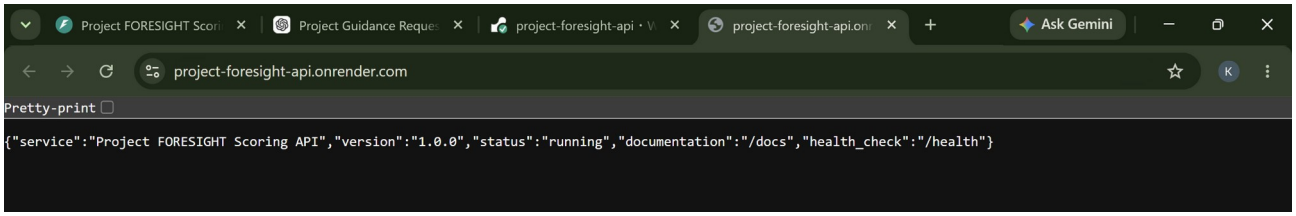

Figure 10. Public Render deployment returning service metadata.

Public endpoints:

API root: https://project-foresight-api.onrender.com/

Health check: https://project-foresight-api.onrender.com/health

Swagger documentation: https://project-foresight-api.onrender.com/docs

13 Testing, Reproducibility, and Governance

Controls that make the project dependable and refreshable.

13.1 Reproducibility

```
python -m src.pipeline
python -m src.features
python -m src.baseline
python -m src.model
python -m src.rolling_cv
python -m src.future_forecast
python -m src.risk_scoring
python -m streamlit run app/dashboard.py
python -m uvicorn service.main:app --reload --port 8000
```

13.2 Testing and validation controls

- Schema, uniqueness, type, missing-value, and range checks in the data pipeline.
- Chronological train, validation, and test partitions instead of random time-series splits.
- Rolling-origin cross-validation to test repeated future windows.
- Baseline comparisons reported alongside the selected model.
- Non-negative clipping for operational demand forecasts.
- API request validation and explicit error responses.
- Manual user testing of dashboard filters, prioritised tables, SKU Explorer, downloads, and public deployment URLs.

13.3 Governance and scope controls

- No automated purchase order is placed by the system.
- Financial values are decision-support estimates derived from dataset prices and costs.
- Risk thresholds and the 1.65 safety-stock multiplier should be reviewed with business owners before production automation.
- The README documents setup, run commands, deliverables, and deployed links.

14 Business Impact and Recommendations

What the forecast and risk engine indicate for the current scoring run.

INR 793,250.16	**INR 6,347.14**	**INR 1,128,180.40**
Potential lost revenue	**Excess inventory value**	**Replenishment cost**
prioritise stockout actions	capital tied in excess	recommended orders
9	**93**	**3**
High stockout SKUs	**Medium stockout SKUs**	**High overstock SKUs**
immediate action	supplier review	pause or clear

14.1 Recommended operating actions

6. Act first on high stockout-risk SKUs ranked by potential lost revenue.
7. Use the MOQ-rounded recommended order quantity as a planning starting point, then validate supplier availability and budget.
8. Review medium stockout-risk SKUs for delayed supply or underestimated demand.
9. Pause or defer orders for high overstock-risk SKUs and consider transfers, promotions, or markdowns.
10. Refresh the pipeline, forecast, and risk score weekly before purchase decisions.
11. Track realised lost sales, service level, excess stock, and forecast bias after each refresh.

Decision principle: Prioritisation combines probability/severity with INR exposure so scarce purchasing and planning attention is directed toward the most financially important SKUs.

15 Limitations and Future Enhancements

Known constraints and practical next steps.

Current limitation	Recommended enhancement
Forecast error varies substantially by SKU.	Add segment-specific models or hierarchical forecasting for intermittent and high-volume products.
Prediction intervals are not included.	Add calibrated 80% and 95% intervals and use them in safety-stock decisions.
Promotion effects are observational.	Add controlled uplift modelling or richer campaign attributes.
External shocks and supplier disruptions are not represented.	Incorporate supplier reliability, market events, weather, and lead-time variability when available.
Risk thresholds are rule-based.	Calibrate thresholds against service-level targets and realised business outcomes.
No drift monitoring.	Track feature drift, forecast bias, WAPE, and risk precision over time.
No automated enterprise integration.	Add scheduled ingestion and authenticated endpoints after governance approval.

These limitations are reported openly because the project is intended as a decision-support system. Human review remains necessary before committing purchasing budget or inventory actions.

16 Conclusion

Final assessment of the delivered project.

Project FORESIGHT satisfies the core business and technical requirements of the engagement. It creates a reproducible path from raw operational files to weekly forecasts, compares a selected model against naive baselines using time-aware validation, produces an 8-week future forecast for all 150 SKUs, converts that forecast into transparent inventory-risk decisions, and exposes the results through a live dashboard and a public API.

The selected HistGradientBoosting model achieved 39.51% final-test WAPE and outperformed both the previous-week and seasonal-naive baselines. The current risk run identifies 9 high stockout-risk SKUs, 3 high overstock-risk SKUs, INR 793,250.16 in potential lost revenue, INR 6,347.14 in excess inventory value, and INR 1,128,180.40 in recommended replenishment cost.

Most importantly, the project does not stop at prediction. It translates model output into prioritised actions that operations and finance stakeholders can understand, review, and use in weekly planning.

Final status: Source code, live Streamlit deployment, live FastAPI deployment, final executive readout, and this written project report are complete. The remaining submission items are the demo video and feedback video prepared by the team.

17 References and Appendices

Project links, report-compliance map, API summary, and glossary.

17.1 References and live links

Zidio Development - Project FORESIGHT: Demand & Inventory Intelligence, Engagement Brief v1.0: Internal engagement specification supplied for the internship.

GitHub repository: https://github.com/kundanad2102-ui/project-foresight

Streamlit dashboard: https://project-foresight-szbpqvrcxp8x6sbrf8qmzx.streamlit.app/

FastAPI service: https://project-foresight-api.onrender.com/

FastAPI Swagger documentation: https://project-foresight-api.onrender.com/docs

17.2 Submission requirement coverage

Submission requirement	Coverage in this report
Project overview	Executive Summary and Sections 1-2.
Technology stack	Section 3.
Architecture	Section 4 and Figure 1.
Screenshots	Sections 8, 11, and 12; Figures 4 and 7-10.
Results and business impact	Sections 8-10 and 14.
Conclusion	Section 16.
Source code and deployment links	Cover page and Section 17.1.

17.3 Glossary

Term	Meaning	Term	Meaning
SKU	One unique product or item.	WAPE	Weighted Absolute Percentage Error; lower is better.
MAE	Mean Absolute Error.	RMSE	Root Mean Squared Error; penalises large errors.
Rolling-origin validation	Repeated testing on later time windows.	Leakage-safe feature	Uses only pre-forecast information.
MOQ	Minimum Order Quantity.	Inventory position	On-hand plus on-order inventory.
API	Application Programming Interface.		